

Workerbee audit v0.01 review

1.3.1

1.3.2

1.3.3

2.1.1

2.1.2

2.1.3

2.1.4

2.1.5

2.1.6

2.1.7

2.1.8

2.1.9

2.2.2

2.2.3

2.3.1

2.3.2

2.3.3

3.1.1

3.1.2

3.1.3

3.1.4

3.1.5

3.2.1

3.2.2

3.2.3

3.3.1.

3.3.2

3.3.3

3.4.1

3.4.2

4.1.1/4.1.2

4.2.1-4.2.5

4.3

5.1.1

5.1.2

5.1.3

5.2.1

5.2.2

5.2.3

5.3.1

5.3.2

5.3.3

5.4.1

5.4.2

5.4.3

6.1.1

6.1.2

6.1.3

6.2.1-6.2.4

6.3.1-6.3.3

7.1.1/7.1.2

7.2

7.3

8

[review by lyh]

1.3.1

所谓“断连恢复痛点被两处‘不可恢复’明文否定”，我的意思不是节点/session因异常/错误失效后不允许持久化并从持久化状态恢复，而是用户如果自己选择删除某项任务/workflow，那么这些东西虽然需要持久化，但是不允许用户再次恢复他自己选择删除的东西了。这个就是个功能，跟断连恢复压根不是一回事。motivation里面提到的断连恢复痛点是指客户端失效后的重连，是从单agent harness角度说的。对于workerbee来说，如果workflow节点出现断连，则应该尝试恢复，不过恢复失败之后的机制还需要其他设计。

1.3.2

Pending List在节点级，任务发射在workflow级别。这是因为逻辑上就是这样啊。一个workflow发射任务，需要从拓扑图入度为0的节点出发，然后这个任务顺着workflow节点流转，最后完成。一个任务本身是一个整体，但是他在不同的节点上有不同的执行阶段。你可以整体上设计一个pending在整个workflow流里面的任务，但是不同节点可以体现任务的不同阶段，这个肯定是要有一个list去容纳的，而且无法pre-define，是一个runtime的容器。

然后context_len这个确实是我没讲清楚，实际上我想做的事是每个节点到了context_len后自动compact一次。这个字段是用户自己填的，可以和agent harness的ctx_len不一样，compact的时候就是 $\min(\text{context_len}, \text{harness_ctx_len})$ 自动compact一次。

其他说的我都承认，确实如此。

1.3.3

[需要重点考虑的核心算法设计]

没有提到语义歧义、节点丢失与重复依赖的问题来自于当前机制的哪一部分，这些问题具体的含义是什么？也没有说明。提出了定义图/有效图/运行时图三个概念，但从未定义这三个概念。“真正问题是“相邻节点”未声明参照工作图还是静态图：前者解释下 activate 是静默空操作、节点永久丢失（WB-22, Blocker）；后者解释下 deactivate 短路边无清理规则、activate 后产生重复依赖（WB-22/WB-28）”，未说明这两个问题的具体现象以

及后果。仅仅否定了当前版本，但没有给出可建设的建议。（例如如何修改完善当前版本机制，或者重新设计机制等。应该如何修改？或如何重新设计？）

2.1.1

这个确实有很多问题和没有描述清楚的地方，该机制需要详细设计。

2.1.2

[需要重点考虑的核心机制设计]

无能力标签其实是feature，不是问题，用户需要自己负责模型的配置和选择，因为能力强弱在很多情况下都是个主观问题。即便有artificial analysis这种软件，那也是对旗舰模型的不同领域的能力的一个总体评价。你怎么保障一个所谓的强模型在任务或场景迁移后依然是第一梯队的？另外无token成本预算，这个也是用户自己控制。context_len的超限行为上文说过了。列表穷尽后的失败确实缺少对应机制，这一点是需要详细设计的。

2.1.3

这个确实，没有统一的接口适配层、操作/指令集，插件接口也没有规范，这一点需要详细设计。

2.1.4

这三种建图路径，在执行时走的是一条路径，只是建图时不是同一条路径。我看这个2.1.4写的意思好像是误解了，他是觉得AI参与建图的两种方式（一种AI生成，另一种AI做Graph Capture）明确具体执行的底层细节吗？这个细节是workflow执行时框架层面提供的啊，不是用户手动完成的。

如果是AI生成，那么需要让助手模型按照Workflow的结构约束他的生成，将他的生成（可能是json文本，或者其他结构）转变为可用的定义图。

如果是AI捕获，那其实就是直接让一个base model执行这个任务，然后这个model自己保存plan/reasoning path，然后由AI助手基于这些文档生成对应的定义图。

而具体执行的时候，什么生成摘要、注入prompt、托管路径，这个是框架本身需要设计和实现的机制，对用户是透明的。

2.1.5

[需要重点考虑的重要机制设计]

这个是我没考虑到的机制，可以拟定proposal具体设计一下。

2.1.6

[需要重点考虑的核心机制设计]

这个问题我已经说过了，再复制一遍：所谓“断连恢复痛点被两处‘不可恢复’明文否定”，我的意思不是节点/session因异常/错误失效后不允许持久化并从持久化状态恢复，而是用户如果自己选择删除某项任务/workflow，那么这些东西虽然需要持久化，但是不允许用户再次恢复他自己选择删除的东西了。这个就是个功能，跟断连恢复压根不是一回事。motivation里面提到的断连恢复痛点是指客户端失效后的重连，是从单agent harness角度来说的。对于workerbee来说，如果workflow节点出现断连，则应该尝试恢复，不过恢复失败之后的机制还需要其他设计。

但是这个ckpt、重连、崩溃恢复，这些确实是要纳入断连恢复的核心机制设计里。

2.1.7

[需要重点考虑的核心机制设计]

通信层是需要具体设计的机制

2.1.8

[需要重点考虑的核心机制设计]

安全问题和版本管理我确实没详细去思考，这个需要具体设计。

2.1.9

[需要重点考虑的核心机制设计]

这部分缺少设计，因为这不是对用户提供的功能。我出具的文档主要是厘清功能。

2.2.2

我的原意是让底层harness的系统工程与模型能力来全权掌控skills里的内容，至于资源约束、监控这些，都下放到底层实例去做了，框架不提供约束。不过这个Harness集成是框架需要做的设计，如何集成应该对用户透明，用户只能负责配置。

2.2.3

跨平台，主要就是针对Win/Linux和MacOS吧，覆盖什么发行版这个需要具体调研商定一下。我说tmux只是一种可能的持久化会话的替代方案，举例子而已，又不一定真靠这个持久化。自研或者借用成熟的托管层方案不错，这个可以具体调研纳入设计。

2.3.1

这个问题已经说过了，断连后从持久化状态恢复跟用户自己选择删除任务或者workflow是两回事。

2.3.2

[需要重点考虑的核心机制设计]

我这个proposal是功能上的proposal，不是design上的，只是略微提了一嘴大概的抽象视图，具体层次抽象设计、算法设计、机制设计，还得后面调研后详细讨论商定。

2.3.3

[需要重点考虑的核心机制设计]

我承认你说的这个自指性缺陷，确实是个问题，后续design proposal时需要详细讨论

3.1.1

[需要重点考虑的核心机制设计]

这部分的设计可能存在问题，需要重新拟定。然后effort/reasoning维度确实应该加入。但根据我的个人使用经验而言，这个应该是和harness绑定的吧？

3.1.2

这个我是指一个逻辑视图，具体实现时可以不按照这个定义去做。后面设计时重新拟定。

3.1.3

[需要重点考虑的核心机制设计]

有道理，这么说得有个图级别的配置数据结构来存储整个定义图/有效图/运行时图。而且这个还得是动态的，因为定义图在编辑时拓扑会改变。有效图在操作时也会 (de)activate。

3.1.4

[需要重点考虑的核心机制设计]

我不看好resources这个字段的设计，并放在元数据中。因为这个需要你自动识别机器上的资源，如果你不做自动识别和监控，就得用户指定。这样节点配置上就会很复杂。但是放在Skills里面就不一样，Skills自由度高，还可以复用。但放到skill确实缺少显式约束，这也是个问题。

或者可以用户如果不配置，则可以选择默认可用所有资源（或者说所有剩余资源，至少不能去kill其他的已经存在的进程），如果配置的话不能让用户自己去写资源配置，至少是做成可以选择的列表，之让用户填写关键数值。

3.1.5

[需要考虑的机制设计]

这块需要按照社区上主流常见的mcp call方案设计并实现。

3.2.1

[需要重点考虑的核心机制设计]

这个你的建议还是可以的，我复制一份原始的想法：Pending List在节点级，任务发射在workflow级别。这是因为逻辑上就是这样啊。一个workflow发射任务，需要从拓扑图入度为0的节点出发，然后这个任务顺着workflow节点流转，最后完成。一个任务本身是一个整体，但是他在不同的节点上有不同的执行阶段。你可以整体上设计一个pending在整个workflow流里面的任务，但是不同节点可以体现任务的不同阶段，这个肯定是要有一个list去容纳的，而且无法pre-define，是一个runtime的容器。

如果你的投影就是这个任务在不同节点流转时的状态，那么没啥问题。持久化可以在 Workflow 级别上。但是这个“可以重复提交任务”却无去重键，我表述不准确，其实我的意思是多次提交任务，task id 肯定是不一样的，而且 task id 是系统生成的，用户顶多重复任务内容。priority 这块，确实不是 workflow 级别调整的，而是节点的投影调整。这个是在节点 pending 的任务投影上做的。

3.2.2

[需要重点考虑的重要机制设计]

有道理，功能上确实可以扩展状态。

3.2.3

我这个不是具体字段设计，一个类几十个字段，还有复合的类（成员本身也是个类实例），我咋设计得完，这个就是让你（AI）设计的，我只是描述我要的功能。

3.3.1.

需要设计阶段拟定。

3.3.2

[需要重点考虑的核心机制设计]

这种扁平的存储结构确实有问题，需要设计阶段重新拟定。

3.3.3

[需要重点考虑的重要机制设计]

可以参考此建议。

3.4.1

[需要重点考虑的核心机制设计]

需要设计阶段拟定。

3.4.2

[需要重点考虑的核心机制设计]

可以参考此建议。

4.1.1/4.1.2

[需要重点考虑的核心算法设计]

无环只是一个必要条件。

4.2.1-4.2.5

[需要重点考虑的核心算法设计]

需要校验是否正确。

4.3

[需要重点考虑的核心算法设计]

需要形式化分析、检查、校验、测试该机制设计是否合理。

5.1.1

有道理，可以参考该建议。

5.1.2

需要参考此建议进行设计。

5.1.3

[需要重点考虑的核心机制设计]

这个提议没有说清楚WorkflowRun对应什么用户行为啊？是提交一个任务到Workflow拓扑图上吗，还是说启动一个Workflow有效图到运行时图？TaskInstance是否应该是一个对用户不可见的字段，因为用户只负责向Workflow提交任务，具体任务在不同节点上的

流转是系统框架自动识别的（拓扑排序）。具体怎么判定失败，失败后应该复用还是向上游汇报然后重新执行，还是说直接废弃，这个需要在具体设计时讨论，但第一版应该设计的不那么复杂。

5.2.1

[需要重点考虑的重要机制设计]

优先级字段这种设计实现时可以具体讨论。发射对象（等效于提交任务）是workflow这个非常正常啊，用户又不会手动去判断拓扑图的排序，他怎么知道一个任务提交进去会先从哪个节点开始，然后流转到哪里去。他只负责搭建图，这种底层操作细节没必要让用户负责。所有权不在节点上啊，节点上只是任务在这个阶段的投影而已，但是我们能调度这个投影改变pending任务的执行顺序，这又不矛盾。历史管理就是提交到这个workflow的所有任务的一个可读可写（主要针对删除/暂停/恢复）的handler列表。

其他建议可以参考。

5.2.2

[需要重点考虑的核心机制设计]

这点说的很到位，并不是串行处理多个提交，实际上是每个节点是一组session，这个组随着提交任务的数量而扩展，但是逻辑上在同一个节点上是串行执行任务的，也就是执行完一个新开一个session放到组里。

5.2.3

[需要重点考虑的核心机制设计]

这个需要设计阶段考虑。

5.3.1

[需要重点考虑的核心机制设计]

5.3.2

[需要重点考虑的核心机制设计]

5.3.3

[需要重点考虑的核心机制设计]

5.4.1

这个问题我上文说了很多遍了，用户自己选择删除workflow或者任务跟这个任务对应的session挂了重连是两回事，用户他自己要删的，没必要给他保留恢复功能。我们只恢复因为异常或错误挂掉的session。

5.4.2

[需要重点考虑的核心机制设计]

5.4.3

[需要重点考虑的核心机制设计]

可以参考此建议进行设计

6.1.1

[需要重点考虑的核心机制设计]

这个得具体考虑一下设计上到底是让哪边去控制资源约束了。

6.1.2

[需要重点考虑的核心机制设计]

异常和错误状态是需要重点考虑的，对应机制也需要设计。

6.1.3

历史数据是为了给用户回显他提交的任务经历了什么流程，只写不读占磁盘这个事情，现有agent harness也这样啊，你不删session/clear或者其他的清理方式，持久化文件就是越积越多。DSH/CC/Codex他们是有啥清理磁盘的进程吗？

6.2.1-6.2.4

[需要重点考虑的核心机制设计]

这部分得重点设计并验证，确保任务完成/删除/系统崩溃等情况下资源正确得到释放。

6.3.1-6.3.3

[需要重点考虑的核心机制设计]

7.1.1/7.1.2

[需要重点考虑的核心机制设计]

7.2

[需要重点考虑的核心机制设计]

7.3

可以参考此建议设计。

8

可以参考此节提供的roadmap进行设计。